

Predicting Occupational Domains from Synthetic Persona Descriptions: An Evaluation of NLP Pipelines

Ryan Ahmed and Farhan Zarif and Md. Sajid Hossain

BRAC University

Dhaka, Bangladesh

{ryan.ahmed, farhan.zarif, md.sajid.hossain1}@g.bracu.ac.bd

Abstract

The task of occupational domain classification from synthetic persona descriptions presents some challenges due to the diverse terminology and sequence length variations. In this paper, we evaluate the performance of 10 different model architectures on a balanced subset of 17,760 rows of the PersonaHub dataset. We compare standard text vectorization techniques against deep contextual embeddings by training Classical Machine Learning models, Recurrent Neural Networks (RNNs), and Transformer-based architectures. Furthermore, an ablation study on data preprocessing was performed to find the differences between the impact of heavy text preprocessing and light text preprocessing on model performance. The results demonstrated that Transformer architectures significantly outperform sequential and classical models. On the other hand, early recurrent architectures such as SimpleRNN showed substantial degradation due to the vanishing gradient problem on extended sequences. Finally, we implement a soft-voting ensemble combining the top-performing diverse architectures, successfully improving upon the BERT baseline.

1 Introduction

Researchers and Companies use Large Language Models (LLMs) to generate large numbers of synthetic datasets of human personas. As the generation process often remains unpredictable, it might naturally generate too many personas with the characteristics of a Software Engineer and too few personas with the characteristics of an Environmental Scientist. Therefore, a professional domain classifier allows researchers to audit these massive datasets and measure the distribution of occupations. Furthermore, such a classifier can be useful for Dynamic Prompt Routing, Multi-Agent System Simulation, and Bias and Fairness Evaluation.

This study uses a part of the elite_persona subset

from the PersonaHub (Chan et al., 2024) dataset. It consists of textual data of persona descriptions, paired with their general domains and specific domains. Only the first 100,000 rows were loaded and only the general domain (top 1 percent) column was used for the research project. The loaded dataset initially had 2353 unique classes but exactly the top 10 distinct occupation classes were selected for the task.

This research aims to answer the following questions:

1. How does the intensity of data preprocessing impact the classification accuracy of classical models compared to Transformers?
2. How do traditional sparse matrix representations compare against dense, sequential word embeddings when identifying occupational terminology?
3. How much do gating mechanisms and bidirectional processing mitigate the vanishing gradient problem observed in standard SimpleRNNs on long text sequences?
4. How sensitive are these various architectures to regularization techniques when preventing overfitting on text data?

2 Related Work

2.1 Persona-Based Text and Domain Classification

Our used dataset, PersonaHub utilizes billions of auto-curated persona descriptions to various perspectives to generate synthetic texts covering different domains. Our task is to classify the domain based on the persona texts. This falls under the category of short-text classification. It is a unique challenge due to informal style and lexical diversity in a short range of text.

2.2 Classical Machine Learning Approaches

Before neural network models became popular, text classification heavily relied on classical machine learning algorithms based on BOW or TF-IDF representations. Logistic regression may not be the first choice for the final model, but it definitely is a strong contender for baseline checking. Naive Bayes is another efficient statistical method based on word frequency used in NLP. Additionally, Random Forest (Breiman, 2001) utilizes decision trees performing well over noisy, high-dimensional features and overfitting. These models were chosen as a lower bound baseline to compare with the performance of more complex models.

2.3 Recurrent Neural Network

Recurrent Neural Networks classifies texts by modeling sequential word dependencies instead of treating those text as an unordered bag of features. SimpleRNN, processes those text token by token which causes vanishing gradients over long sequences of tokens which gives the idea for LSTM (Hochreiter and Schmidhuber, 1997) and GRUs (Cho et al., 2014a). They both use specialized gates to regulate information flow and capture long sequences of tokens and capture the context efficiently. They have a bidirectional variants which makes the inputs in both forward and backward directions leveraging complete left and right context making them more powerful pre-transformer sequence classifiers.

2.4 Transformer based Approach

Transformers like BERT (Devlin et al., 2019) modernized text classification by completely replacing recurrence with self-attention mechanism and with their ability of pre-training and replaceable backbone architecture they boost model accuracy. Unlike RNNs, BERT encodes bidirectional context for every token at the same time which allows to capture more context in depth and transfer those general purpose language representations. Fine tuning BERT models achieves consistent higher state of the art results which sometimes acts as the benchmark against classical and recurrent architectures.

3 Methodology

3.1 Dataset

The elite_persona subset of the PersonaHub Dataset consists of textual data of persona descriptions, paired with their respective ‘general domain

(top 1 percent)’, ‘general domain (top 0.1 percent)’, ‘specific domain (top 1 percent)’ and ‘specific domain (top 0.1 percent)’ columns. First, 100,000 rows of the part10 of the elite_persona subset were loaded. Only the ‘persona’ and ‘general domain (top 1 percent)’ columns were kept for the research project. After data cleaning and balancing, the final dataset used for this research consists of 17,760 total rows evenly distributed across exactly 10 distinct occupation classes.

We visualize the differences in occupational terminology for History and Medicine in Figure 1 (see Appendix A for the word clouds for all 10 classes).

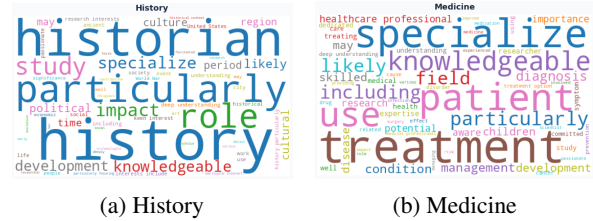


Figure 1: A comparison of occupational terminology showing distinct vocabulary between History and Medicine.

An initial exploratory data analysis was done to examine the distribution of instances across different classes. As illustrated in Figure 2, the initial dataset revealed significant class imbalance. While majority classes contained a large number of persona descriptions, minority classes such as Public Health were significantly underrepresented, requiring data balancing.

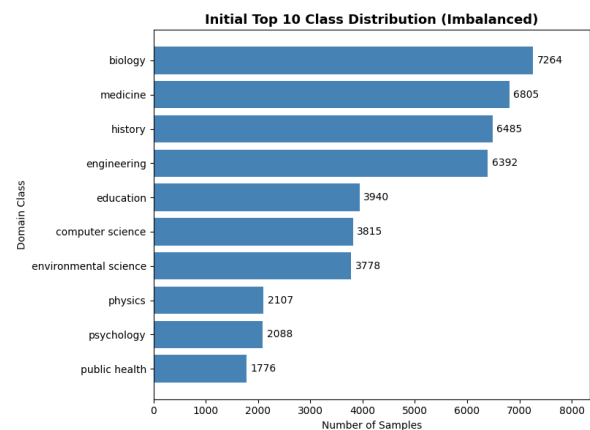


Figure 2: Initial instance counts for the top 10 occupation classes, demonstrating heavy class imbalance.

We analyzed the sequence lengths of the persona descriptions, as text length can impact the vanishing gradient problem of recurrent architectures. Overall, the documents contained an aver-

age of 92.86 words ($SD = 26.24$, $min = 11.00$, $max = 463.00$). To identify potential length-based class imbalances, sequence lengths were examined across the 10 chosen domains. As shown in Figure 3, the word count distribution remains relatively consistent across all occupations. The exact per-class statistical breakdown and overall length distribution histogram are provided in Appendix A.

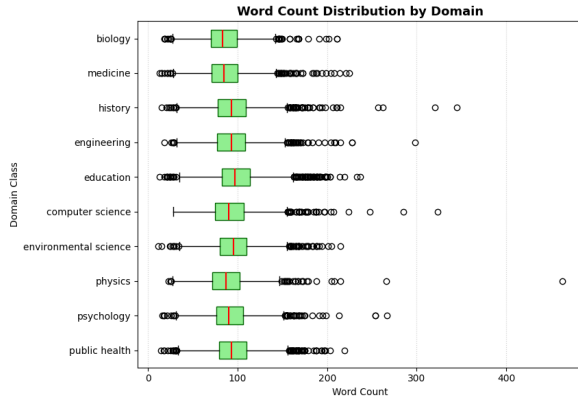


Figure 3: Word count distribution for the top 10 occupation classes.

3.2 Preprocessing

The initial subset of the PersonaHub dataset contained 100000 rows and 6 columns. We only kept the ‘persona’ and ‘general domain (top 1 percent)’ columns as they were not relevant for our classification task. In both columns, the search for null values returned 0. However, we found 16768 rows containing only the string ‘None’ in different representations in the ‘general domain (top 1 percent)’ column which were removed. There were no occurrences of duplicate texts. 116 rows were removed due to having non-English text.

In the ‘general domain (top 1 percent)’ column we found a total of 2353 unique classes. As per our classification task, we identified the top 10 classes from the column and only kept respective rows. Figure 2 shows the top 10 classes name and their counts in the dataset. To fix the class imbalance, we used random sampling based on lowest class count (public health: 1776) so that every class has the same amount of data. Then, we encoded the classes to numerical values using label encoder. The data was splitted into 70 : 15 : 15 ratio sets for train, validation and test.

In data preprocessing and tokenization, we took two different approaches for the training dataset. The first approach was heavy preprocessing where

the text data was lowercased, normalized whitespace, removed punctuation and digits, removed stopwords and lemmatized. The second approach, light preprocessing, only consists of whitespace normalization. Table 1 shows samples of original, heavy and light preprocessing.

3.3 Word Representations

In this study, five different types of representations techniques were used and for each representation both heavy and light data preprocessing were applied.

TF-IDF is a word representation technique where the representation is based on the importance of a certain word. A word appearing frequently in a particular document indicates importance of the word whereas the word being used in all documents in corpus indicates it as a common word, thus unimportant. We have used this technique with the hyperparameter maximum features as 5000. This approach, however, fails to capture the context from the neighbor words.

Given a word, Word2Vec’s (Mikolov et al., 2013) skip-gram model attempts to predict surrounding words by learning the relationship of the adjacent word inside the context window by storing them in a vector from the training data. In our implementation, we allocated 100 sized vectors to represent each word with a context window of 5. The words that had less frequency than 2 were excluded. Word2Vec captures local context but can not capture global context due to context window constraint.

GloVe (Pennington et al., 2014) is an unsupervised model pretrained on large text corpora that can capture long contexts by focusing on the chances of two words appearing together from a co-occurrence matrix. Here, we have used the glove.6B.100d model from the official repository. However, it does not understand the positional contextual meaning.

We tokenized words from train, validation and test with Tokenizer having vocabulary size of 10000. The maximum length of each document was chosen as 150 as the mean of the word count in all classes is around 100. Document lengths greater than that were truncated and lesser than were post-padded. These tokens were used to pre-compute embedding matrices of 100 sized vectors.

BERT is a pretrained encoder-only architecture model that can hold bidirectional context. We tokenized train, validation and test data using

Original Text	Heavy Preprocessing	Light Preprocessing
A mental health professional who specializes i...	mental health professional specializes working...	A mental health professional who specializes i...
A scientist who specializes in environmental h...	scientist specializes environmental health tox...	A scientist who specializes in environmental h...
A psychotherapist who uses narrative therapy t...	psychotherapist us narrative therapy help indi...	A psychotherapist who uses narrative therapy t...

Table 1: A comparative example of the heavy and light text tokenization applied to persona instances.

bert-base-uncased AutoTokenizer with a maximum length of 150. Later, we used these tokens in the BERT model.

3.4 Model Architectures

We combined different preprocessing and representations techniques to train and evaluate four different models first. Then we use selective different hyperparameters to experiment with ten different models. For the hyperparameter tuning experiments, we kept the best resulting preprocessing and representation approach and tuned other parameters.

At first, we trained logistic regression with heavy and light preprocessing on TF-IDF, Word2Vec and GloVe vectors. Later in hyperparameter tuning, for baseline, we used regularization hyperparameter as 1, ridge regularization and default optimization algorithm. Then, we experimented with a high regularization coefficient with lasso regularization and a low regularization coefficient with ridge regularization. All of the experiments were done with a maximum iteration of 1000. For the hyperparameter tuning experiments, we only worked with the best performing word representation and preprocessing which was TF-IDF with heavy preprocessing.

Second, we chose LSTM because of its combination of long short term memory gated architecture that remembers important information from the text. We used heavy and light preprocessing of Word2Vec and GloVe representation one by one. In the model, we passed the precomputed embedding matrix weights with each word’s vector size of 100. We kept these embedding weights untrainable and masked the padding as zero value. Then we passed these to the LSTM of 64 neurons with 0.2 dropout. Finally, these are sent to the output layer with softmax activation. We used sparse cat-

egorical crossentropy for multiclass classification with Adam (Kingma and Ba, 2014) optimizer. We trained the model for 10 epochs with a batch size of 64. In this case, the best combination was GloVe representation and heavy preprocessing which was used in tuning experiments. In hyperparameter tuning, we used the same in baseline and then experimented with 128 unit LSTM layer with dropout of 0.3 and 32 units of LSTM layer with dropout of 0.5. The experiments were conducted with 5 epochs.

Finally, we trained the BERT (bert-base-uncased) model on the heavy and then light preprocessed tokens created by BERT’s own tokenizer. We trained the model for 3 epochs with a batch size of 16. The light preprocessing tokens performed better, so we proceeded to tune other hyperparameters with this. We experimented with learning rate $2e-5$ with 16 batch size, then learning rate $5e-5$ with 32 batch size and learning rate $3e-5$ with additional weight decay parameter of 0.01 and 16 batch size. All the experiments were run for 3 epochs.

Additionally, we tried tuning on a few other models for comparison. We ran the Random Forest classifier model on the TF-IDF representation of the heavy preprocessing for this experiment. For baseline, we took `n_estimators` as 100 with null `max_depth`. Then we tested with a large forest with `n_estimators` as 200 and 50 `max_depth` restriction. Lastly, we tried a fast 50 `n_estimators` with 10 minimum samples split pruning.

We used the same preprocessing and representation condition with Naive Bayes model. In this model, we experimented with different alpha values to compare the results. In baseline, we used the alpha value as 1.0. In tuning, we tried low smoothing ($\alpha = 0.1$) and high smoothing ($\alpha = 10.0$).

Next, we tried tuning SimpleRNN. For baseline,

we used the pre-computed GloVe representation of heavy preprocessed tokens with each word’s vector size as 100. We skipped training of this embedding and set the paddings as zero. This embedding is then passed onto a 64 unit SimpleRNN with 0.2 dropout. Finally, we used softmax activation for multiclass classification. For the same reason, we used sparse categorical loss. We chose Adam optimizer with 0.001 learning rate. In hyperparameter tuning, we experimented with the SimpleRNN unit number and dropout rates. Our high capacity configuration was of 128 SimpleRNN units with 0.3 dropout and lightweight configuration was of 32 units with 0.5 dropout. Each of the configurations were trained for 5 epochs. Notice that these hyperparameter tuning configurations match with LSTM that was discussed above. This was done intentionally because they are structurally close and under the same family of Recurrent Neural Network.

For the same reason, we experimented with the same configurations with other models under the same family of RNN. First, GRU outperforms LSTM with faster computation and less memory usage because of its more efficient gated architecture. While simple architecture only looks or contains information of the left side of the currently processing word, the context of a word may be related to the next words which simple architectures fail to capture. With bidirectionality, the models are provided with the context of both left and right words which makes the context capturing more accurate. So, we also experimented with bidirectional SimpleRNN, bidirectional GRU and bidirectional LSTM. In all of these experiments, the matrix embedding, baseline parameters, high capacity and light capacity parameters, number of epochs, batch size, loss function and optimizer were kept the same as the ones used in SimpleRNN.

4 Results and Comparative Analysis

4.1 Metrics

We report accuracy and weighted F1. On a perfectly balanced test set the two coincide except when per class recall is highly uneven. All reported per-class figures are computed from the test-set confusion matrices.

4.2 Preprocessing Interacts with Representation

Table 2 reports validation performance for every combination of representation and preprocessing

Table 2: Effect of preprocessing across representations on the validation set.

Model	Representation	Heavy F1	Light F1
Logistic Regression	TF-IDF	0.9161	0.9089
Logistic Regression	Word2Vec	0.9004	0.8839
Logistic Regression	GloVe	0.8819	0.8838
LSTM	Word2Vec	0.9185	0.8468
LSTM	GloVe	0.9142	0.9095
BERT Base	WordPiece	0.9430	0.9513

strategy. The effect of preprocessing depends not only on the dataset, but also on how each representation constructs its vocabulary.

Three regularities emerge, first representation whose vocabulary is learned from the training corpus benefit most from aggressive normalization: Word2Vec + LSTM pairing gains 7.2 F1 points, because 12,432 documents are insufficient to estimate reliable vectors for inflected forms and high frequency function words. Secondly, pre trained GloVe is effective invariant to the regime as the distributional information that stop word removal discards is already encoded in its 6B token pre training. Thirdly, in the opposite direction we find that only BERT that prefers light preprocessing. It’s subword vocabulary, positional embeddings and pre training objective all presuppose well formed running text and if we were to remove the function words and punctuations then it destroys precisely the syntactic scaffolding that the self attention mechanism uses.

We therefore, fix heavy pre-processing for all classical and recurrent models and light preprocessing for BERT in all subsequent experiments.

4.3 Hyperparameter Sensitivity

Appendix C reports all 30 configurations. Sensitivity is non uniform across families; the gated recurrent models and BERT span at most and 2.2 and 1.5 points respectively.

The largest single effect in the study is L1 regularization on Logistic Regression, which costs 10.2 accuracy points. Domain The second largest is the SimpleRNN’s 11.9 point spread, which is anti monotonic in capacity.

In Table 3 we see that the BERT model training loss falls to a very marginalized state by the third epoch. Checkpoint selection is driven by validation loss rather than accuracy, so two of the three configurations are restored to an earlier and less accurate state than their epoch-3 peak. only $lr = 5e - 5$, $bs = 32$ attains its minimum loss and

maximum accuracy which is why the checkpoint is carried forward to testing.

Configuration	Epoch 1	Epoch 2	Epoch 3
lr=2e-5, bs=16	0.9414	0.9497	0.9561
lr=5e-5, bs=32	0.9268	0.9452	0.9531
lr=3e-5, bs=16, wd=0.01	0.9381	0.9418	0.9512

Table 3: BERT fine-tuning dynamics on the validation set.

Table 4 reports test-set performance for the selected configuration of each family of models. All ten models exceed the 10% random baseline; nine of ten exceed 0.87 weighted F1.

BERT base attains the highest results leading the strongest recurrent model by 2.0 points and the strongest classical model by 3.1 points. Accuracy and weighted F1 agree to within 0.001 for every model except SimpleRNN whose 2.0 point divergence (0.4872 vs 0.4672) already signals uneven per class recall.

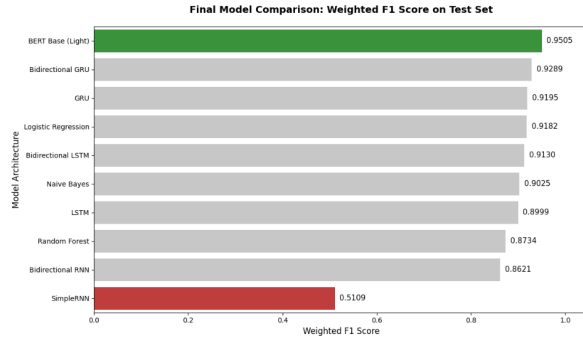


Figure 4: Weighted F1 score on Test Set

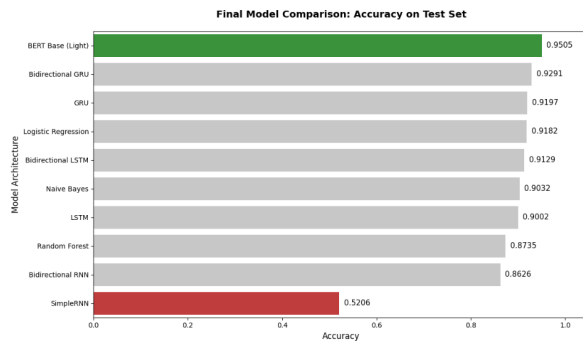


Figure 5: Accuracy score on Test Set

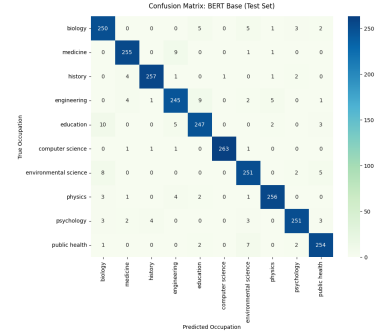
4.4 Per-Class Behaviour

Table 5 reports per class F1 for four representation systems spanning the performance stage. The ordering is broadly stable across models. History is the easiest class for every system without exception

and Biology and Engineering are the hardest for nearly every system.

4.5 Confusion Matrix: Best vs Worst

Figure 6 contrasts the test-set confusion matrices of the best- and worst-performing systems. In both matrices, each cell reports the raw number of predictions, with darker colors indicating larger counts.



(a) BERT Base (best-performing system)



(b) SimpleRNN (worst-performing system)

Figure 6: Test-set confusion matrices for the best- and worst-performing systems.

BERT Base. The diagonal is uniformly dominant, containing between 245 and 263 correct predictions out of 266–267 instances per class, and no individual off-diagonal cell exceeds 10 errors. The largest residual confusions are Education → Biology (10), Medicine → Engineering (9), Engineering → Education (9), Environmental Science → Biology (8), and Public Health → Environmental Science (7). Overall, these errors remain sparse compared with the strong concentration of predictions along the diagonal.

SimpleRNN. The failure is qualitatively different from mere weakness. The model partially collapses onto a small set of attractor classes: Physics and Public Health absorb far more predictions than expected. Both classes achieve approximately 0.68

Rank	Model	Representation / Preproc.	Accuracy	Weighted F1	Δ
1	BERT Base	Word2Vec / light	0.9493	0.9494	—
2	Bidirectional GRU	GloVe / heavy	0.9294	0.9292	−2.02
3	Bidirectional LSTM	GloVe / heavy	0.9249	0.9248	−2.46
4	GRU	GloVe / heavy	0.9230	0.9230	−2.64
5	Logistic Regression	TF-IDF / heavy	0.9182	0.9182	−3.12
6	LSTM	GloVe / heavy	0.9103	0.9103	−3.91
7	Naive Bayes	TF-IDF / heavy	0.9032	0.9025	−4.69
8	Bidirectional SimpleRNN	GloVe / heavy	0.8739	0.8735	−7.59
9	Random Forest	TF-IDF / heavy	0.8735	0.8734	−7.60
10	SimpleRNN	GloVe / heavy	0.4872	0.4672	−48.22

Table 4: Main results. Test-set performance of the selected configuration per model family ($n = 2,664$; 266–267 documents per class).

Class	BERT	Bi-GRU	LogReg	SimpleRNN	Mean (10 models)
history	0.991	0.976	0.989	0.790	0.959
education	0.972	0.944	0.945	0.553	0.900
computer science	0.957	0.921	0.930	0.550	0.884
physics	0.961	0.943	0.938	0.500	0.883
public health	0.951	0.939	0.915	0.518	0.876
medicine	0.935	0.925	0.874	0.521	0.854
psychology	0.954	0.943	0.918	0.308	0.853
environmental science	0.929	0.905	0.906	0.507	0.851
engineering	0.921	0.891	0.890	0.312	0.810
biology	0.924	0.907	0.878	0.114	0.801
Macro average	0.949	0.929	0.918	0.467	—

Table 5: Per-class test F1 for four representative systems.

recall, but their precision falls to 0.39 and 0.42, respectively. Biology deteriorates to 0.06 recall, with only 17 of 266 documents correctly recovered and 80 incorrectly predicted as Physics. The largest confusions—Psychology → Environmental Science (84), Psychology → Public Health (82), Environmental Science → Public Health (80), and Biology → Physics (80)—are roughly an order of magnitude larger than BERT’s largest off-diagonal cell.

4.6 Ensemble Implementation and Benchmark Improvement

To further optimize classification performance and improve upon the established benchmark results, we implemented an ensemble combining the best performing models from the 3 distinct architectural families: Logistic Regression, Bidirectional GRU, and BERT Base. Because these models process textual features in different ways, we assumed that their combined predictions would reduce individual model biases and improve on edge case misclassifications. The ensemble was constructed using a soft-voting mechanism and the final predicted class $\hat{y}$ was determined by averaging these probability distributions and selecting the class with the maximum mean probability. The BERT Base model

established a highly accurate baseline benchmark, achieving an accuracy of 95.05% on the test set. But, by implementing the soft-voting ensemble, we successfully achieved a new peak accuracy of 95.35%.

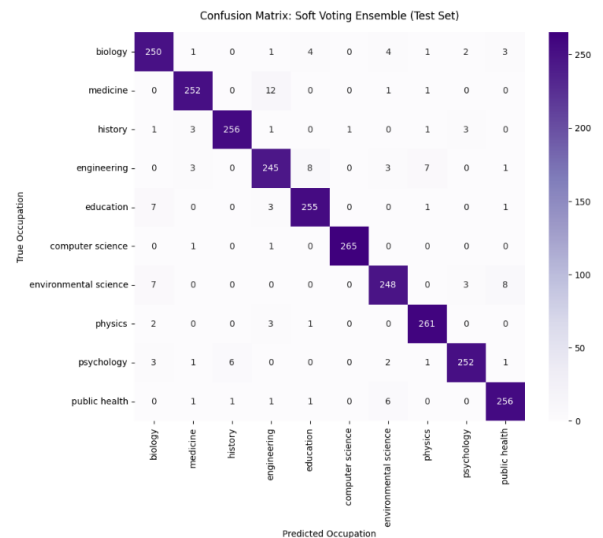


Figure 7: Confusion Matrix of the Soft Voting Ensemble

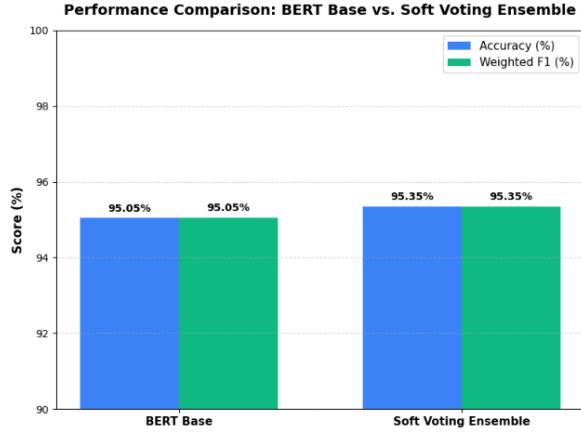


Figure 8: Performance Comparison between BERT Base and Soft Voting Ensemble

5 Conclusion

5.1 Key Takeaways

This study evaluated 10 distinct machine learning architectures for the task of occupational domain classification using textual persona descriptions. The findings from the study prove that deep contextual embeddings do better than classical algorithms and early recurrent networks. BERT achieved an F1 score of 95.05%, demonstrating a superior ability to extract complex occupational semantics. On the other hand, the poor performance of the SimpleRNN architecture of 48.7% highlights the impact of the vanishing gradient problem when processing extended textual sequences. Furthermore, preprocessing and vectorization ablation studies revealed that heavy preprocessing performs better for classical ML models and surprisingly for RNNs. Paired with TF-IDF vectorization, the classical models achieved highly competitive results. For the recurrent architectures, GloVe and Word2Vec embeddings performed almost the same, though GloVe had a marginally better contextual representation. In contrast, BERT achieved its peak performance using the light preprocessing pipeline, demonstrating that aggressive text normalization destroys the syntactical structure and bidirectional context that Transformer architectures rely on. Finally, the soft-voting ensemble successfully improved upon the BERT baseline, proving that combining these diverse architectures returns the best predictive performance.

5.2 Limitation

Despite strong classification results, this study has notable limitations. To reduce severe class imbal-

ance and prevent model bias, the dataset was under-sampled to match the minority class. Consequently, the models were trained on only 17,760 total instances across 10 domains, representing only a fraction of the 2,353 unique occupations present in the original dataset. Another limitation was all of the models except the BERT Base was run for only 5 epochs due to computation limitation, the BERT Base was run for 3 epochs. Additionally, the performance improvements of the ensemble and BERT models comes at a significant computational cost compared to models like Logistic Regression which still maintained a highly competitive F1 score.

5.3 Future Improvements

Future research should focus on scaling the classification framework to cover a broader set of occupational domains. Moreover, future iterations could implement synthetic data generation techniques like SMOTE to handle extreme class imbalances instead of relying on random undersampling. Lastly, future studies should research with more advanced transformer variants, like RoBERTa, to further improve the benchmark accuracy.

References

- Martín Abadi et al. 2015. [Tensorflow: Large-scale machine learning on heterogeneous distributed systems](#). Software available from tensorflow.org.
- Alfred V. Aho and Jeffrey D. Ullman. 1972. *The Theory of Parsing, Translation and Compiling*, volume 1. Prentice-Hall, Englewood Cliffs, NJ.
- American Psychological Association. 1983. *Publications Manual*. American Psychological Association, Washington, DC.
- Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6:1817–1853.
- Galen Andrew and Jianfeng Gao. 2007. Scalable training of L1-regularized log-linear models. In *Proceedings of the 24th International Conference on Machine Learning*, pages 33–40.
- Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media, Inc.
- Leo Breiman. 2001. [Random forests](#). *Machine Learning*, 45(1):5–32.
- Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu. 2024. [Scaling Synthetic Data Creation with 1,000,000,000 Personas](#). *arXiv preprint*. ArXiv:2406.20094 [cs].
- Ashok K. Chandra, Dexter C. Kozen, and Larry J. Stockmeyer. 1981. [Alternation](#). *Journal of the Association for Computing Machinery*, 28(1):114–133.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014a. [Learning phrase representations using RNN encoder-decoder for statistical machine translation](#). In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar. Association for Computational Linguistics.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014b. [Learning phrase representations using RNN encoder-decoder for statistical machine translation](#). In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734. Association for Computational Linguistics.
- François Chollet et al. 2015. Keras. <https://keras.io>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: Pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, volume 1, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
- Dan Gusfield. 1997. *Algorithms on Strings, Trees and Sequences*. Cambridge University Press, Cambridge, UK.
- Charles R Harris et al. 2020. Array programming with NumPy. *Nature*, 585(7825):357–362.
- Sepp Hochreiter and Jürgen Schmidhuber. 1997. [Long short-term memory](#). *Neural Computation*, 9(8):1735–1780.
- John D Hunter. 2007. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.
- Diederik P Kingma and Jimmy Ba. 2014. [Adam: A method for stochastic optimization](#). *arXiv preprint arXiv:1412.6980*.
- Quentin Lhoest et al. 2021. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 175–184.
- Andrew McCallum and Kamal Nigam. 1998. A comparison of event models for naive bayes text classification. In *Proceedings of the AAAI-98 Workshop on Learning for Text Categorization*, pages 41–48.
- Wes McKinney. 2010. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, pages 56 – 61.
- Tomas Mikolov, Kai Chen, Greg Corrado, and Ilya Sutskever. 2013. [Efficient estimation of word representations in vector space](#). *arXiv preprint arXiv:1301.3781*.
- Andreas Mueller. 2023. word_cloud: A little word cloud generator in python. https://github.com/amueller/word_cloud.
- Fabian Pedregosa et al. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. [GloVe: Global vectors for word representation](#). In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543. Association for Computational Linguistics.
- Mohammad Sadegh Rasooli and Joel R. Tetreault. 2015. [Yara parser: A fast and accurate dependency parser](#). *Computing Research Repository*, arXiv:1503.06733. Version 2.

- Radim Řehůřek and Petr Sojka. 2010. Software Framework for Topic Modelling with Large Corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, pages 45–50.
- Mike Schuster and Kuldip K. Paliwal. 1997. [Bidirectional recurrent neural networks](#). *IEEE Transactions on Signal Processing*, 45(11):2673–2681.
- Pauli Virtanen et al. 2020. SciPy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272.
- Michael L Waskom. 2021. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021.
- Thomas Wolf et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

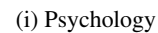
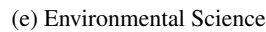
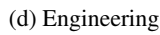
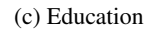
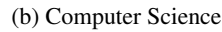
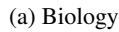
[illegible]

Figure 9: Complete word cloud distributions across all 10 occupation classes.

Domain	Count	Min	Max	Mean	Median	Std
Biology	1776	18	211	85.19	83.0	22.64
Computer Science	1776	28	323	93.14	90.0	26.82
Education	1776	13	237	100.04	97.0	29.08
Engineering	1776	18	298	94.16	93.0	25.28
Environmental Science	1776	11	215	96.90	95.0	25.03
History	1776	15	345	95.40	93.0	27.54
Medicine	1776	13	225	87.16	85.0	24.94
Physics	1776	23	463	88.64	87.0	25.59
Psychology	1776	16	267	92.38	90.0	25.25
Public Health	1776	14	219	95.59	93.0	26.08

Table 6: Per-class document length statistics (word count) across the 10 selected occupation domains.

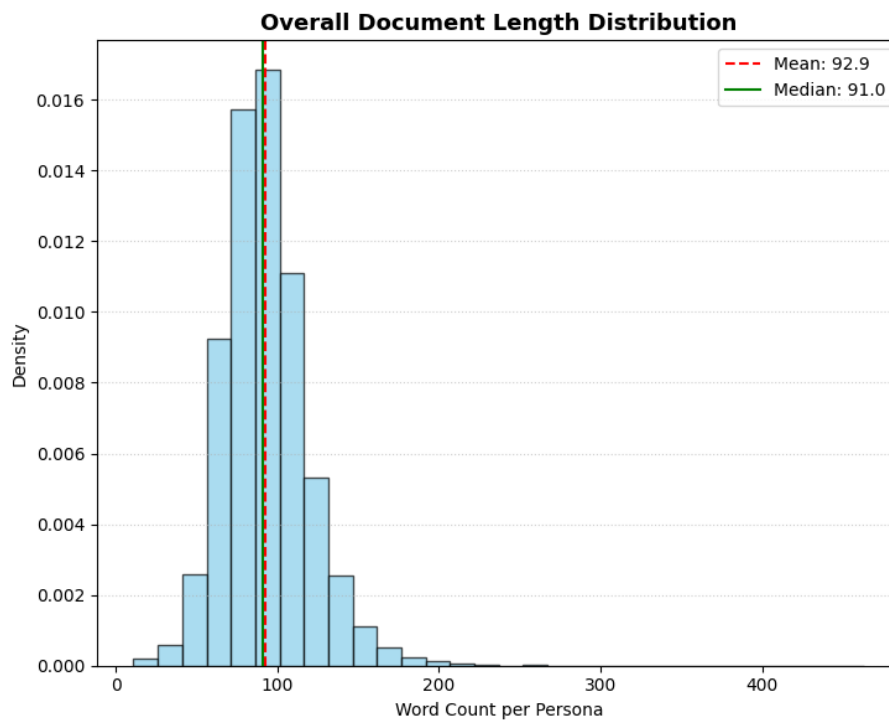


Figure 10: Overall document length distribution (word count) across the entire balanced dataset.

B Data Preprocessing and Text Representation Ablation Study

This appendix contains the ablation study of Heavy and Light data preprocessing paired with TF-IDF, GloVe and Word2Vec text representation techniques.

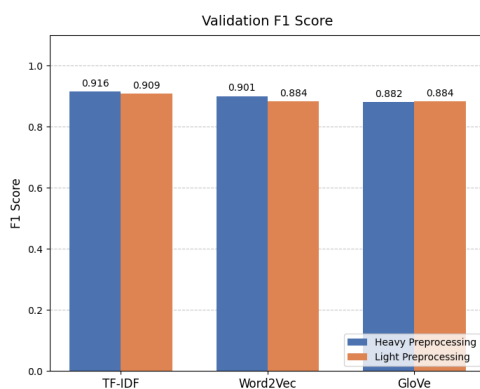


Figure 11: Ablation study on Logistic Regression

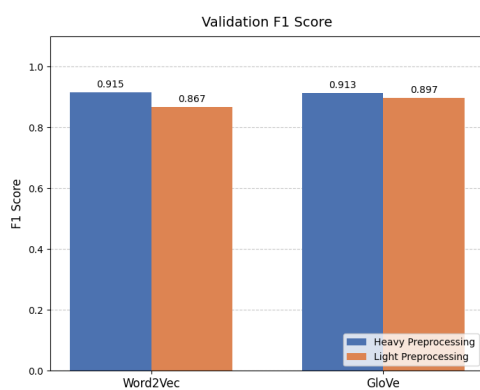


Figure 12: Ablation study on LSTM

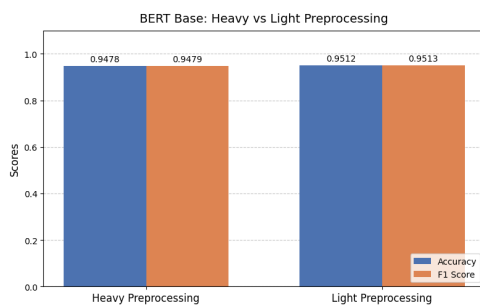


Figure 13: Ablation study on BERT Base

C Validation performance of all 30 configurations

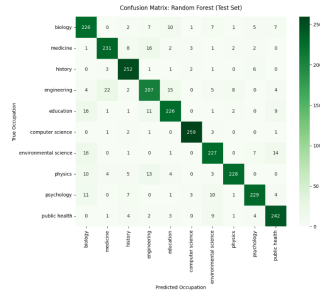
This appendix contains the complete set of validation performance of all 30 configurations.

Model	Configuration	Accuracy	F1	Spread $\uparrow$
Random Forest	n_estimators=100, max_depth=None	0.8844	0.8842	1.31
Random Forest	n_estimators=200, max_depth=50	0.8836	0.8834	
Random Forest	n_estimators=50, min_samples_split=10	0.8716	0.8711	
Logistic Regression	C=1.0, 12, lbfgs	0.9159	0.9161	9.99
Logistic Regression	C=0.1, 11, saga	0.8146	0.8166	
Logistic Regression	C=10.0, 12, lbfgs	0.9163	0.9165	
Naive Bayes	alpha=1.0	0.8904	0.8899	0.52
Naive Bayes	alpha=0.1	0.8851	0.8847	
Naive Bayes	alpha=10.0	0.8866	0.8856	
SimpleRNN	units=64, dropout=0.2	0.5533	0.5493	11.88
SimpleRNN	units=128, dropout=0.3	0.4505	0.4305	
SimpleRNN	units=32, dropout=0.5	0.5023	0.4837	
GRU	units=64, dropout=0.2	0.9140	0.9145	2.21
GRU	units=128, dropout=0.3	0.9249	0.9250	
GRU	units=32, dropout=0.5	0.9028	0.9029	
LSTM	units=64, dropout=0.2	0.9133	0.9135	1.80
LSTM	units=128, dropout=0.3	0.9167	0.9169	
LSTM	units=32, dropout=0.5	0.8994	0.8989	
Bi-SimpleRNN	units=64, dropout=0.2	0.8690	0.8682	2.20
Bi-SimpleRNN	units=128, dropout=0.3	0.8840	0.8851	
Bi-SimpleRNN	units=32, dropout=0.5	0.8619	0.8631	
Bi-GRU	units=64, dropout=0.2	0.9294	0.9295	0.96
Bi-GRU	units=128, dropout=0.3	0.9268	0.9268	
Bi-GRU	units=32, dropout=0.5	0.9197	0.9199	
Bi-LSTM	units=64, dropout=0.2	0.9080	0.9082	1.36
Bi-LSTM	units=128, dropout=0.3	0.9219	0.9218	
Bi-LSTM	units=32, dropout=0.5	0.9148	0.9151	
BERT Base	lr=2e-5, bs=16	0.9501	0.9501	1.47
BERT Base	lr=5e-5, bs=32	0.9531	0.9531	
BERT Base	lr=3e-5, bs=16, wd=0.01	0.9384	0.9384	

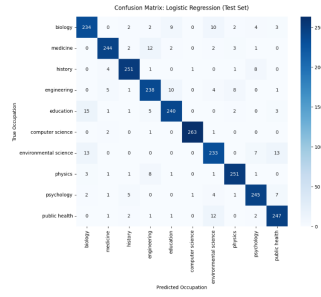
Table 7: Validation performance of all 30 configurations. Bold marks the selected configuration within each family; $\uparrow$ marks the intra-family spread.

D Confusion Matrices of the Models

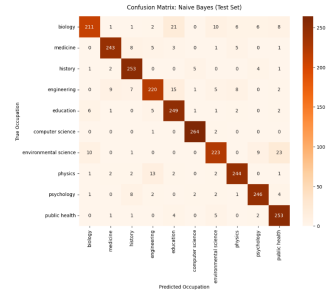
This appendix contains the complete set of confusion matrices generated for all 10 models.



(a) Random Forest



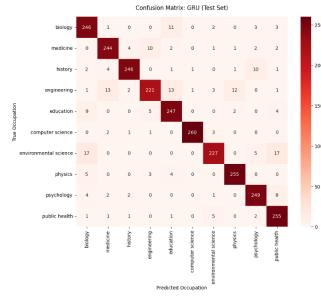
(b) Logistic Regression



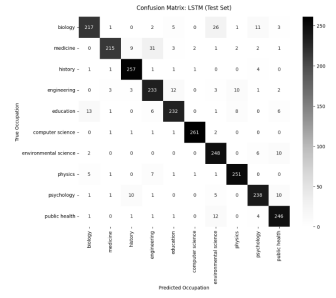
(c) Naive Bayes



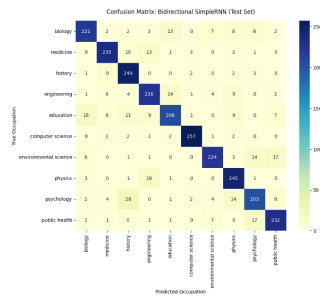
(d) SimpleRNN



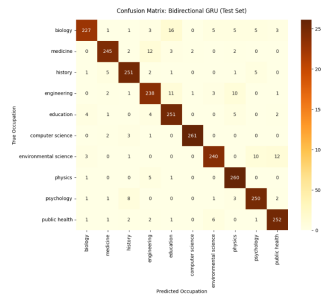
(e) GRU



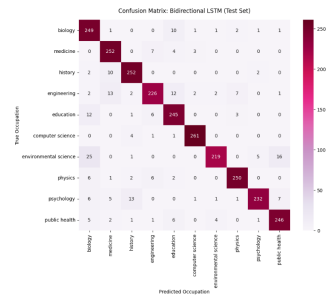
(f) LSTM



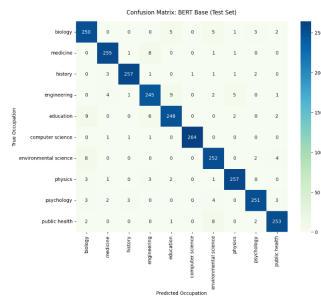
(g) Bidirectional SimpleRNN



(h) Bidirectional GRU



(i) Bidirectional LSTM



(j) BERT Base

Figure 14: Confusion matrices for all 10 evaluated models.